Submissions

Problem	Language	Time	Result	Score	Action
Detect HTML Tags, Attributes and Attribute Values	pypy3	about 8 hours	Accepted	30.0	View results
Print Function	pypy3	about 8 hours	Accepted	20.0	View results
Write a function	pypy3	about 8 hours	Accepted	10.0	View results
Write a function	pypy3	about 8 hours	Accepted	10.0	View results
Loops	pypy3	12 days	Accepted	10.0	View results
Python: Division	pypy3	12 days	Accepted	10.0	View results
Python: Division	pypy3	12 days	Accepted	10.0	View results
Arithmetic Operators	pypy3	12 days	Accepted	10.0	View results
Python If-Else	pypy3	12 days	Accepted	10.0	View results
Say "Hello, World!" With Python	pypy3	12 days	Accepted	5.0	View results

‹ 1 ... 8 9 10 11 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
Word Order	pypy3	about 8 hours	Accepted	50.0	View results
Word Order	pypy3	about 8 hours	Accepted	50.0	View results
Collections.OrderedDict()	pypy3	about 8 hours	Accepted	20.0	View results
Collections.namedtuple()	pypy3	about 8 hours	Accepted	20.0	View results
DefaultDict Tutorial	pypy3	about 8 hours	Accepted	20.0	View results
Finding the percentage	pypy3	about 8 hours	Accepted	10.0	View results
Nested Lists	pypy3	about 8 hours	Accepted	10.0	View results
Find the Runner-Up Score!	pypy3	about 8 hours	Accepted	10.0	View results
List Comprehensions	pypy3	about 8 hours	Accepted	10.0	View results
Validating UID	pypy3	about 8 hours	Accepted	40.0	View results

‹ 1 ... 8 9 10 11 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
Iterables and Iterators	pypy3	about 7 hours	Accepted	40.0	View results
Triangle Quest	pypy3	about 7 hours	Accepted	20.0	View results
Triangle Quest	pypy3	about 7 hours	Accepted	20.0	View results
Integers Come In All Sizes	pypy3	about 7 hours	Accepted	10.0	View results
Power - Mod Power	pypy3	about 7 hours	Accepted	10.0	View results
Mod Divmod	pypy3	about 7 hours	Accepted	10.0	View results
Triangle Quest 2	pypy3	about 7 hours	Accepted	20.0	View results
Company Logo	pypy3	about 8 hours	Accepted	30.0	View results
Piling Up!	pypy3	about 8 hours	Accepted	50.0	View results
Collections.deque()	pypy3	about 8 hours	Accepted	20.0	View results

‹ 1 ... 8 9 10 11 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
String Split and Join	pypy3	about 3 hours	Accepted	10.0	View results
sWAP cASE	pypy3	about 3 hours	Accepted	10.0	View results
Tuples	pypy3	about 7 hours	Accepted	10.0	View results
Lists	pypy3	about 7 hours	Accepted	10.0	View results
Class 2 - Find the Torsional Angle	pypy3	about 7 hours	Accepted	20.0	View results
Python If-Else	pypy3	about 7 hours	Accepted	10.0	View results
Inner and Outer	pypy3	about 7 hours	Accepted	20.0	View results
Dot and Cross	pypy3	about 7 hours	Accepted	20.0	View results
Mean, Var, and Std	pypy3	about 7 hours	Accepted	20.0	View results
Maximize It!	pypy3	about 7 hours	Accepted	50.0	View results

‹ 1 ... 8 9 10 11 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
Capitalize!	pypy3	about 3 hours	Accepted	20.0	View results
Alphabet Rangoli	pypy3	about 3 hours	Accepted	20.0	View results
String Formatting	pypy3	about 3 hours	Accepted	10.0	View results
Designer Door Mat	pypy3	about 3 hours	Accepted	10.0	View results
Text Wrap	pypy3	about 3 hours	Accepted	10.0	View results
Text Alignment	pypy3	about 3 hours	Accepted	10.0	View results
String Validators	pypy3	about 3 hours	Accepted	10.0	View results
Find a string	pypy3	about 3 hours	Accepted	10.0	View results
Mutations	pypy3	about 3 hours	Accepted	10.0	View results
What's Your Name?	pypy3	about 3 hours	Accepted	10.0	View results

‹ 1 ... 7 8 9 ... 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
Set .union() Operation	pypy3	about 2 hours	Accepted	10.0	View results
Set .discard(), .remove() & .pop()	pypy3	about 2 hours	Accepted	10.0	View results
Set .add()	pypy3	about 2 hours	Accepted	10.0	View results
ginortS	pypy3	about 2 hours	Accepted	40.0	View results
Any or All	pypy3	about 2 hours	Accepted	20.0	View results
Athlete Sort	pypy3	about 2 hours	Accepted	30.0	View results
Python Evaluation	pypy3	about 2 hours	Accepted	20.0	View results
Input()	pypy3	about 2 hours	Accepted	20.0	View results
Merge the Tools!	pypy3	about 2 hours	Accepted	40.0	View results
The Minion Game	pypy3	about 2 hours	Accepted	40.0	View results

‹ 1 ... 6 7 8 ... 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
HTML Parser - Part 1	pypy3	about 2 hours	Accepted	30.0	View results
Hex Color Code	pypy3	about 2 hours	Accepted	30.0	View results
Validating and Parsing Email Addresses	pypy3	about 2 hours	Accepted	20.0	View results
Check Strict Superset	pypy3	about 2 hours	Accepted	10.0	View results
Check Subset	pypy3	about 2 hours	Accepted	10.0	View results
The Captain's Room	pypy3	about 2 hours	Accepted	10.0	View results
Set Mutations	pypy3	about 2 hours	Accepted	10.0	View results
Set .symmetric_difference() Operation	pypy3	about 2 hours	Accepted	10.0	View results
Set .difference() Operation	pypy3	about 2 hours	Accepted	10.0	View results
Set .intersection() Operation	pypy3	about 2 hours	Accepted	10.0	View results

‹ 1 ... 5 6 7 ... 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
Eye and Identity	pypy3	about 2 hours	Accepted	20.0	View results
Zeros and Ones	pypy3	about 2 hours	Accepted	20.0	View results
Concatenate	pypy3	about 2 hours	Accepted	20.0	View results
Validating phone numbers	pypy3	about 2 hours	Accepted	20.0	View results
collections.Counter()	pypy3	about 2 hours	Accepted	10.0	View results
Reduce Function	pypy3	about 2 hours	Accepted	30.0	View results
Matrix Script	pypy3	about 2 hours	Accepted	100.0	View results
Validating Postal Codes	pypy3	about 2 hours	Accepted	80.0	View results
Validating Credit Card Numbers	pypy3	about 2 hours	Accepted	40.0	View results
HTML Parser - Part 2	pypy3	about 2 hours	Accepted	30.0	View results

‹ 1 ... 4 5 6 ... 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
itertools.combinations_with_replacement()	pypy3	about 2 hours	Accepted	10.0	View results
itertools.combinations()	pypy3	about 2 hours	Accepted	10.0	View results
itertools.permutations()	pypy3	about 2 hours	Accepted	10.0	View results
itertools.product()	pypy3	about 2 hours	Accepted	10.0	View results
Linear Algebra	pypy3	about 2 hours	Accepted	20.0	View results
Polynomials	pypy3	about 2 hours	Accepted	20.0	View results
Min and Max	pypy3	about 2 hours	Accepted	20.0	View results
Sum and Prod	pypy3	about 2 hours	Accepted	20.0	View results
Floor, Ceil and Rint	pypy3	about 2 hours	Accepted	20.0	View results
Array Mathematics	pypy3	about 2 hours	Accepted	20.0	View results

‹ 1 2 3 4 5 ... 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
Time Delta	python3	about 1 hour	Accepted	30.0	View results
Transpose and Flatten	pypy3	about 1 hour	Accepted	20.0	View results
Validating Roman Numerals	pypy3	about 1 hour	Accepted	20.0	View results
Regex Substitution	pypy3	about 1 hour	Accepted	20.0	View results
Re.start() & Re.end()	pypy3	about 1 hour	Accepted	20.0	View results
Re.findall() & Re.finditer()	pypy3	about 1 hour	Accepted	20.0	View results
Group(), Groups() & Groupdict()	pypy3	about 1 hour	Accepted	20.0	View results
Re.split()	pypy3	about 1 hour	Accepted	20.0	View results
Detect Floating Point Number	pypy3	about 1 hour	Accepted	20.0	View results
Compress the String!	pypy3	about 1 hour	Accepted	20.0	View results

‹ 1 2 3 4 5 ... 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
Arrays	pypy3	about 1 hour	Accepted	20.0	View results
XML2 - Find the Maximum Depth	pypy3	about 1 hour	Accepted	20.0	View results
Zipped!	pypy3	about 1 hour	Accepted	10.0	View results
Incorrect Regex	pypy3	about 1 hour	Accepted	20.0	View results
Exceptions	pypy3	about 1 hour	Accepted	10.0	View results
Calendar Module	pypy3	about 1 hour	Accepted	10.0	View results
Decorators 2 - Name Directory	pypy3	about 1 hour	Accepted	30.0	View results
No Idea!	pypy3	about 1 hour	Accepted	50.0	View results
Symmetric Difference	pypy3	about 1 hour	Accepted	10.0	View results
Introduction to Sets	pypy3	about 1 hour	Accepted	10.0	View results

‹ 1 2 3 4 5 ... 12 ›

Submissions

Problem	Language	Time	Result	Score	Action
Say "Hello, World!" With Python	pypy3	23 minutes	Accepted	5.0	View results
Polar Coordinates	pypy3	33 minutes	Accepted	10.0	View results
Default Arguments	pypy3	34 minutes	Accepted	30.0	View results
Words Score	pypy3	36 minutes	Accepted	10.0	View results
Standardize Mobile Number Using Decorators	pypy3	about 1 hour	Accepted	30.0	View results
XML 1 - Find the Score	pypy3	about 1 hour	Accepted	20.0	View results
Map and Lambda Function	pypy3	about 1 hour	Accepted	20.0	View results
Validating Email Addresses With a Filter	pypy3	about 1 hour	Accepted	20.0	View results
Zipped!	pypy3	about 1 hour	Accepted	10.0	View results
Shape and Reshape	pypy3	about 1 hour	Accepted	20.0	View results

1 2 3 4 5 ... 12

Birthday Cake Candles

20 more points to get your first star!
Rank: 4361244 | Points: 10/30

Your Birthday Cake Candles submission got 10.00 points. Condivic Post

You are now 20 points away from the 1st star for your problem solving badge.

Try the next challenge | Try a Random Challenge

You are in charge of the cake for a child's birthday. It will have one candle for each year of their total age. They will only be able to blow out the tallest of the candles. Your task is to count how many candles are the tallest.

Example

$candles = [4, 4, 1, 3]$

The tallest candles are 4 units high. There are 2 candles with this height, so the function should return 2.

Function Description

Complete the function $birthdayCakeCandles$ with the following parameter(s):

- $int\ candles[n]$: the candle heights

Returns

- int: the number of candles that are tallest

Input Format

The first line contains a single integer, n, the size of $candles[]$.

The second line contains n space-separated integers, where each integer i describes the height of $candles[i]$.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq candles[i] \leq 10^7$

Sample Input 0

```
4
3 2 1 3
```

Sample Output 0

```
2
```

Explanation 0

Candle heights are $[3, 2, 1, 3]$. The tallest candles are 3 units, and there are 2 of them.

Change Theme Language Pypy 3

```
import math
import os
import random
import re
import sys

#
# Complete the 'birthdayCakeCandles' function below.
#
# The function is expected to return an INTEGER.
# The function accepts INTEGER_ARRAY candles as parameter.
#

def birthdayCakeCandles(candles):
    return candles.count(max(candles))
if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    candles_count = int(input().strip())

    candles = list(map(int, input().rstrip().split()))

    result = birthdayCakeCandles(candles)

    fptr.write(str(result) + '\n')

    fptr.close()

```

Line: 30 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

You have earned 10.00 points!

You are now 20 points away from the 1st star for your problem solving badge.

33% 10/30



Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Compiler Message

Success

Input (stdin) Download

```
4
3 2 1 3
```

Test case 4

Test case 5

Test case 6

Expected Output

Download

```
1 2
```

Number Line Jumps

10 more points to get your first star!

Rank: 3780110 | Points: 20/30

Your Number Line Jumps submission got 10.00 points. Condivic Post

You are now 10 points away from the 1st star for your problem solving badge.

Try the next challenge | Try a Random Challenge

Problem Submissions Leaderboard Editorial

You are choreographing a circus show with various animals. For one act, you are given two kangaroos on a number line ready to jump in the positive direction (i.e, toward positive infinity).

- The first kangaroo starts at location $x1$ and moves at a rate of $v1$ meters per jump.
- The second kangaroo starts at location $x2$ and moves at a rate of $v2$ meters per jump.

You have to figure out a way to get both kangaroos at the same location at the same time as part of the show. If it is possible, return YES, otherwise return NO.

Example

$x1 = 2$

$v1 = 1$

$x2 = 1$

$v2 = 2$

After one jump, they are both at $x = 3$, ($x1 + v1 = 2 + 1$, $x2 + v2 = 1 + 2$), so the answer is YES.

Function Description

Complete the function kangaroo in the editor below.

kangaroo has the following parameter(s):

- int x1, int v1: starting position and jump distance for kangaroo 1
- int x2, int v2: starting position and jump distance for kangaroo 2

Returns

- string: either YES or NO

Input Format

A single line of four space-separated integers denoting the respective values of $x1$, $v1$, $x2$, and $v2$.

Constraints

- $0 \leq x1 < x2 \leq 10000$
- $1 \leq v1 \leq 10000$
- $1 \leq v2 \leq 10000$

Sample Input 0

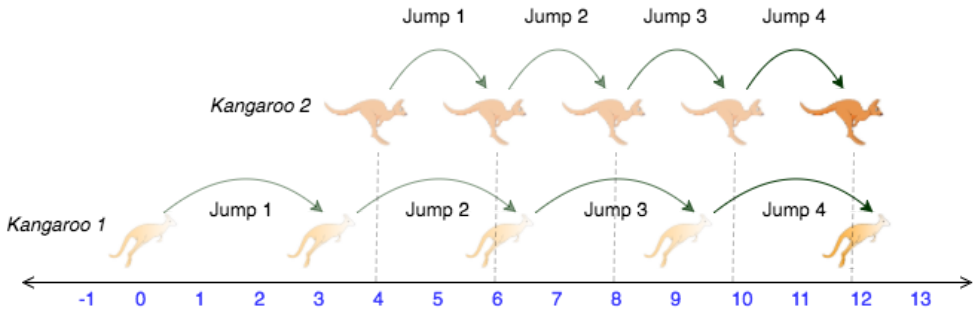
```
0 3 4 2
```

Sample Output 0

YES

Explanation 0

The two kangaroos jump through the following sequence of locations:



From the image, it is clear that the kangaroos meet at the same location (number 12 on the number line) after same number of jumps (4 jumps), and we print YES.

Sample Input 1

```
0 2 5 3
```

Sample Output 1

```
NO
```

Explanation 1

The second kangaroo has a starting location that is ahead (further to the right) of the first kangaroo's starting location (i.e., $x_2 > x_1$). Because the second kangaroo moves at a faster rate (meaning $v_2 > v_1$) and is already ahead of the first kangaroo, the first kangaroo will never be able to catch up. Thus, we print NO.

Change Theme Language Pypy 3

```
#  2. INTEGER v1
#  3. INTEGER x2
#  4. INTEGER v2
#

def kangaroo(x1, v1, x2, v2):
    if v1 > v2 and (x2 - x1) % (v1 - v2) == 0:
        return 'YES'
    return 'NO'
if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    first_multiple_input = input().rstrip().split()

    x1 = int(first_multiple_input[0])

    v1 = int(first_multiple_input[1])

    x2 = int(first_multiple_input[2])

    v2 = int(first_multiple_input[3])
```

```
36
37      result = kangaroo(x1, v1, x2, v2)
38
39      fptr.write(result + '\n')
40
41      fptr.close()
42
```

Line: 23 Col: 16

Upload Code as File | Test against custom input | Run Code | Submit Code

You have earned 10.00 points!

You are now 10 points away from the 1st star for your problem solving badge.

67% 20/30



Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

- Test case 0
- Test case 1
- Test case 2
- Test case 3
- Test case 4
- Test case 5
- Test case 6

Compiler Message

```
Success
```

Input (stdin) Download

```
1  0 3 4 2
```

Expected Output Download

```
1  YES
```

Recursive Digit Sum

65 more points to get your next star!
Rank: 3115516 | Points: 35/100

We define super digit of an integer x using the following rules:

Given an integer, we need to find the super digit of the integer.

- If x has only 1 digit, then its super digit is x.
- Otherwise, the super digit of x is equal to the super digit of the sum of the digits of x.

For example, the super digit of 9875 will be calculated as:

```
super_digit(9875)       9+8+7+5 = 29
super_digit(29)     2 + 9 = 11
super_digit(11)         1 + 1 = 2
super_digit(2)          = 2
```

Example

$n =' 9875'$

$k = 4$

The number p is created by concatenating the string n k times so the initial $p = 9875987598759875$.

```
superDigit(p) = superDigit(9875987598759875)
              9+8+7+5+9+8+7+5+9+8+7+5+9+8+7+5 = 116
superDigit(p) = superDigit(116)
              1+1+6 = 8
superDigit(p) = superDigit(8)
```

All of the digits of p sum to 116. The digits of 116 sum to 8. 8 is only one digit, so it is the super digit.

Function Description

Complete the function superDigit in the editor below. It must return the calculated super digit as an integer.

superDigit has the following parameter(s):

- string n: a string representation of an integer
- int k: the times to concatenate n to make p

Returns

- int: the super digit of n repeated k times

Input Format

The first line contains two space separated integers, n and k.

Constraints

- $1 \leq n < 10^{100000}$
- $1 \leq k \leq 10^5$

Sample Input 0

```
148 3
```

Sample Output 0

```
3
```

Explanation 0

Here $n = 148$ and $k = 3$, so $p = 148148148$.

```
super_digit(P) = super_digit(148148148)
            = super_digit(1+4+8+1+4+8+1+4+8)
            = super_digit(39)
            = super_digit(3+9)
            = super_digit(12)
            = super_digit(1+2)
            = super_digit(3)
            = 3
```

Sample Input 1

```
9875 4
```

Sample Output 1

```
8
```

Sample Input 2

```
123 3
```

Sample Output 2

```
9
```

Explanation 2

Here $n = 123$ and $k = 3$, so $p = 123123123$.

```
super_digit(P) = super_digit(123123123)
            = super_digit(1+2+3+1+2+3+1+2+3)
            = super_digit(18)
            = super_digit(1+8)
            = super_digit(9)
            = 9
```

Change Theme Language Pypy 3

```
 8
 9  #
10  # Complete the 'superDigit' function below.
11  #
12  # The function is expected to return an INTEGER.
13  # The function accepts following parameters:
14  #  1. STRING n
15  #  2. INTEGER k
16  #
17
```

```
18  def superDigit(n, k):
19      total = sum(int(d) for d in n) * k
20      if total < 10:
21          return total
22      return superDigit(str(total), 1)
23
24  if __name__ == '__main__':
25      fptr = open(os.environ['OUTPUT_PATH'], 'w')
26
27      first_multiple_input = input().rstrip().split()
28
29      n = first_multiple_input[0]
30
31      k = int(first_multiple_input[1])
32
33      result = superDigit(n, k)
34
35      fptr.write(str(result) + '\n')
```

Line: 22 Col: 37

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

- Sample Test case 0
- Sample Test case 1
- Sample Test case 2

Input (stdin) Download

```
1 148 3
```

Your Output (stdout)

```
1 3
```

Expected Output Download

```
1 3
```

Viral Advertising

65 more points to get your next star!
Rank: 3115516 | Points: 35/100
Problem Solving

Your Viral Advertising submission got 15.00 points. Condivic Post
You are now 65 points away from the 2nd star for your problem solving badge.
Try the next challenge | Try a Random Challenge

HackerLand Enterprise is adopting a new viral advertising strategy. When they launch a new product, they advertise it to exactly 5 people on social media.

On the first day, half of those 5 people (i.e., $floor(\frac{5}{2}) = 2$) like the advertisement and each shares it with 3 of their friends. At the beginning of the second day, $floor(\frac{5}{2}) \times 3 = 2 \times 3 = 6$ people receive the advertisement.

Each day, $floor(\frac{recipients}{2})$ of the recipients like the advertisement and will share it with 3 friends on the following day. Assuming nobody receives the advertisement twice, determine how many people have liked the ad by the end of a given day, beginning with launch day as day 1.

Example

$n = 5$.

Day	Shared	Liked	Cumulative
1	5	2	2
2	6	3	5
3	9	4	9
4	12	6	15
5	18	9	24

The progression is shown above. The cumulative number of likes on the 5th day is 24.

Function Description

Complete the viralAdvertising function in the editor below.

viralAdvertising has the following parameter(s):

- int n: the day number to report

Returns

- int: the cumulative likes at that day

Input Format

A single integer, n, the day number.

Constraints

- $1 \leq n \leq 50$

Sample Input

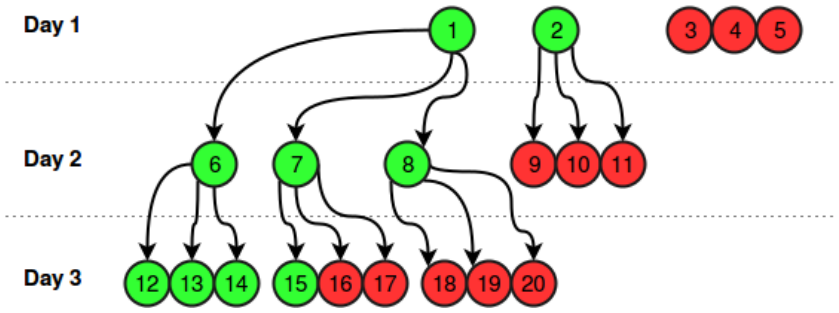
```
3
```

Sample Output

9

Explanation

This example is depicted in the following diagram:



2 people liked the advertisement on the first day, **3** people liked the advertisement on the second day and **4** people liked the advertisement on the third day, so the answer is $\mathbf{2+3+4=9}$.

Change Theme Language Pypy 3

```
#!/bin/python3

import math
import os
import random
import re
import sys

#
# Complete the 'viralAdvertising' function below.
#
# The function is expected to return an INTEGER.
# The function accepts INTEGER n as parameter.
#

def viralAdvertising(n):
    shared = 5
    cumulative = 0
    for _ in range(n):
        liked = shared // 2
        cumulative += liked
        shared = liked * 3
    return cumulative
if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    n = int(input().strip())
```

Line: 23 Col: 22

Upload Code as File Test against custom input Run Code Submit Code

You have earned 15.00 points!

You are now 65 points away from the 2nd star for your problem solving badge.

7% 35/100



Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

- Test case 0
- Test case 1
- Test case 2
- Test case 3
- Test case 4

Compiler Message

```
Success
```

Input (stdin) Download

```
1 3
```

Expected Output Download

```
1 9
```

Insertion Sort - Part 2

5 more points to get your next star!
Rank: 2067179 | Points: 95/100
Problem Solving

Your Insertion Sort - Part 2 submission got 30.00 points. Condivic Post
You are now 5 points away from the 2nd star for your problem solving badge.
Try the next challenge | Try a Random Challenge

In Insertion Sort Part 1, you inserted one element into an array at its correct sorted position. Using the same approach repeatedly, can you sort an entire array?

Guideline: You already can place an element into a sorted array. How can you use that code to build up a sorted array, one element at a time? Note that in the first step, when you consider an array with just the first element, it is already sorted since there's nothing to compare it to.

In this challenge, print the array after each iteration of the insertion sort, i.e., whenever the next element has been inserted at its correct position. Since the array composed of just the first element is already sorted, begin printing after placing the second element.

Example.

$n = 7$

$arr = [3, 4, 7, 5, 6, 2, 1]$

Working from left to right, we get the following output:

```
3 4 7 5 6 2 1
3 4 7 5 6 2 1
3 4 5 7 6 2 1
3 4 5 6 7 2 1
2 3 4 5 6 7 1
1 2 3 4 5 6 7
```

Function Description

Complete the insertionSort2 function in the editor below.

insertionSort2 has the following parameter(s):

- int n: the length of arr
- int arr[n]: an array of integers

Prints

At each iteration, print the array as space-separated integers on its own line.

Input Format

The first line contains an integer, n, the size of arr.

The next line contains n space-separated integers $arr[i]$.

Constraints

$1 \leq n \leq 1000$

$-10000 \leq arr[i] \leq 10000, 0 \leq i < n$

Output Format

Print the entire array on a new line at every iteration.

Sample Input

```
STDIN         Function
-----         --------
6             n = 6
1 4 3 5 6 2    arr = [1, 4, 3, 5, 6, 2]
```

Sample Output

```
1 4 3 5 6 2
1 3 4 5 6 2
1 3 4 5 6 2
1 3 4 5 6 2
1 2 3 4 5 6
```

Explanation

Skip testing $\mathbf{1}$ against itself at position $\mathbf{0}$. It is sorted.

Test position $\mathbf{1}$ against position $\mathbf{0}$: $\mathbf{4} > \mathbf{1}$, no more to check, no change.

Print $\boldsymbol{arr}$

Test position $\mathbf{2}$ against positions $\mathbf{1}$ and $\mathbf{0}$:

- $\mathbf{3} < \mathbf{4}$, new position may be $\mathbf{1}$. Keep checking.
- $\mathbf{3} > \mathbf{1}$, so insert $\mathbf{3}$ at position $\mathbf{1}$ and move others to the right.

Print $\boldsymbol{arr}$

Test position $\mathbf{3}$ against positions $\mathbf{2, 1, 0}$ (as necessary): no change.

Print $\boldsymbol{arr}$

Test position $\mathbf{4}$ against positions $\mathbf{3, 2, 1, 0}$: no change.

Print $\boldsymbol{arr}$

Test position $\mathbf{5}$ against positions $\mathbf{4, 3, 2, 1, 0}$, insert $\mathbf{2}$ at position $\mathbf{1}$ and move others to the right.

Print $\boldsymbol{arr}$

Change Theme Language Pypy 3

```
#!/bin/python3

import math
import os
import random
import re
import sys

#
# Complete the 'insertionSort2' function below.
#
# The function accepts following parameters:
#  1. INTEGER n
#  2. INTEGER_ARRAY arr
#

def insertionSort2(n, arr):
    for i in range(1, n):
        e = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > e:
```

```
22              arr[j + 1] = arr[j]
23              j -= 1
24          arr[j + 1] = e
25          print(*arr)
26  if __name__ == '__main__':
27      n = int(input().strip())
```

Line: 25 Col: 20

Upload Code as File | Test against custom input | Run Code | Submit Code

You have earned 30.00 points!

You are now 5 points away from the 2nd star for your problem solving badge.

93% 95/100



Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

- Test case 0
- Test case 1
- Test case 2
- Test case 3
- Test case 4
- Test case 5

Compiler Message

Success

Input (stdin) Download

```
6
1 4 3 5 6 2
```

Expected Output Download

```
1 4 3 5 6 2
1 3 4 5 6 2
1 3 4 5 6 2
1 3 4 5 6 2
1 2 3 4 5 6
```

Insertion Sort - Part 1

35 more points to get your next star!

Rank: 2468732 | Points: 65/100

Problem Solving

Your Insertion Sort - Part 1 submission got 30.00 points. Condivic Post

You are now 35 points away from the 2nd star for your problem solving badge.

Try the next challenge | Try a Random Challenge

Problem Submissions Leaderboard

Sorting

One common task for computers is to sort data. For example, people might want to see all their files on a computer sorted by size. Since sorting is a simple problem with many different possible solutions, it is often used to introduce the study of algorithms.

Insertion Sort

These challenges will cover Insertion Sort, a simple and intuitive sorting algorithm. We will first start with a nearly sorted list.

Insert element into sorted list

Given a sorted list with an unsorted number e in the rightmost cell, can you write some simple code to insert e into the array so that it remains sorted?

Since this is a learning exercise, it won't be the most efficient way of performing the insertion. It will instead demonstrate the brute-force method in detail.

Assume you are given the array $arr = [1, 2, 4, 5, 3]$ indexed $0 \ldots 4$. Store the value of $arr[4]$. Now test lower index values successively from 3 to 0 until you reach a value that is lower than $arr[4]$, at $arr[1]$ in this case. Each time your test fails, copy the value at the lower index to the current index and print your array. When the next lower indexed value is smaller than $arr[4]$, insert the stored value at the current index and print the entire array.

Example

$n = 5$

$arr = [1, 2, 4, 5, 3]$

Start at the rightmost index. Store the value of $arr[4] = 3$. Compare this to each element to the left until a smaller value is reached. Here are the results as described:

```
1 2 4 5 5
1 2 4 4 5
1 2 3 4 5
```

Function Description

Complete the insertionSort1 function in the editor below.

insertionSort1 has the following parameter(s):

- n: an integer, the size of arr
- arr: an array of integers to sort

Returns

- None: Print the interim and final arrays, each on a new line. No return value is expected.

Input Format

The first line contains the integer n, the size of the array arr.

The next line contains n space-separated integers $arr[0] \ldots arr[n-1]$.

Constraints

$1 \leq n \leq 1000$

$-10000 \leq arr[i] \leq 10000$

Output Format

Print the array as a row of space-separated integers each time there is a shift or insertion.

Sample Input

```
5
2 4 6 8 3
```

Sample Output

```
2 4 6 8 8
2 4 6 6 8
2 4 4 6 8
2 3 4 6 8
```

Explanation

3 is removed from the end of the array.

In the 1st line $8 > 3$, so 8 is shifted one cell to the right.

In the 2nd line $6 > 3$, so 6 is shifted one cell to the right.

In the 3rd line $4 > 3$, so 4 is shifted one cell to the right.

In the 4th line $2 < 3$, so 3 is placed at position 1.

Next Challenge

In the next Challenge, we will complete the insertion sort.

Change Theme Language Pypy 3

```
#!/bin/python3

import math
import os
import random
import re
import sys

#
# Complete the 'insertionSort1' function below.
#
# The function accepts following parameters:
#  1. INTEGER n
#  2. INTEGER_ARRAY arr
#

def insertionSort1(n, arr):
    e = arr[-1]
    i = n - 2
    while i >= 0 and arr[i] > e:
        arr[i + 1] = arr[i]
        print(*arr)
        i -= 1
    arr[i + 1] = e
    print(*arr)
if __name__ == '__main__':
    n = int(input().strip())
```

Line: 25 Col: 16

Upload Code as File | Test against custom input | Run Code | Submit Code

You have earned 30.00 points!

You are now 35 points away from the 2nd star for your problem solving badge.

50% 65/100



Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

- Test case 0
- Test case 1
- Test case 2
- Test case 3

Compiler Message

```
Success
```

Input (stdin) Download

```
1 5
2 2 4 6 8 3
```

Expected Output Download

```
1 2 4 6 8 8
2 2 4 6 6 8
3 2 4 4 6 8
4 2 3 4 6 8
```